

OOP 朋辈辅学

Lec2 构造、析构、拷贝

钟志昊

1. 练习

下方的代码能够正常输出吗？

```
class Outer{
public:
    Outer(int x): x(x){}
    struct Inner{
        void display(Outer& outer){cout << outer.x << endl;}
    };
    Inner inner;
private:
    int x;
};

int main(){
    Outer outer(10);
    outer.inner.display(outer);
    return 0;
}
```

1、 在以下代码片段中， x 的最终值是多少？

```
int x = 100;  
auto& y = x;  
y = 200;
```

2. 如果用户输入 Hello World, 执行 `string s; cin >> s;`后, 变量 s 的内容是？

3. 以下关于指针和引用的说法正确的是？

A. 指针和引用都可以为空 B. 指针和引用都必须有初始值

C. 引用的本质是对象的别名 D. 指针和引用都可以被重新赋值

2. 函数

2.1 传参方式

C++的函数声明/定义和 C 是相同的，此处不赘述重复内容。

C++的函数传参有三种方式：

- 值传递：创建参数的副本。函数内部修改不会影响原变量
- 指针传递：传递变量的地址。可以通过指针修改原变量
- 引用传递：传递变量的别名。效率高且能修改原变量，在 C++ 中更推荐用引用，而不是指针

对于大型对象，如 `std::vector`，基本是要使用引用传递，否则需要承受拷贝的时间和空间开销，如果不需要修改，可以使用常量引用，如 `void func(const std::vector<int>& v)`

2.2 默认参数

默认参数(Default Arguments)允许在函数声明或定义时为参数指定一个默认值。如果调用函数时未提供该参数的值，编译器会自动使用预设的默认值。

```
void printSum(int a, int b=10){  
    cout << a+b << endl;  
}
```

```
int main(){  
    printSum(5); // 输出15  
    printSum(5, 5); // 输出10  
    return 0;  
}
```

2.3 函数重载

C++允许定义多个同名但参数列表不同（类型或个数）的函数

```
void print(int i) {cout << "Integer" << endl;}
void print(double f) {cout << "Double" << endl;}
void print(int i, double f) {cout << "Integer & Double" <<
endl;}
int main(){
    int a=1;
    double b=2.0;
    print(a);
    print(b);
    print(a, b);
    return 0;
}
```

2.3 函数重载

但是现在有个问题，按照上一页那三个函数的定义，下面这段代码会输出什么？

```
float c=3.0;  
print(c);
```

那下面这段代码能正确输出吗？

```
long d=4;  
print(d);
```

2.4 内联函数

对于代码量极小且频繁调用的函数，可以使用 `inline` 关键字，它**建议**编译器在调用处直接展开代码（类似于宏定义），以减少函数调用的开销。注意只是建议，实际是否内联由编译器决定。

```
inline int getMax(int a, int b){  
    return a>b?a:b;  
}
```

2.5 匿名函数

匿名函数顾名思义是没有函数名的函数，C++中对应的语法也被称作 Lambda 表达式，它允许你在代码中定义内联函数，而无需显式声明一个函数。其完整形式为：

```
[capture](parameters)->return_type{body}
```

- [capture] (捕获列表)：定义 Lambda 可以访问外部作用域中的哪些变量
- (parameters) (参数列表)：与普通函数参数一致
- ->return_type (返回类型)：尾置返回类型，通常可以省略，由编译器自动推导
- {body} (函数体)：具体的实现逻辑

2.5 匿名函数

常用的捕获列表如下:

- `[]`: 不捕获, 不能使用外部任何非静态局部变量
- `[x]`: 按值捕获变量 `x`
- `[&x]`: 按引用捕获变量 `x`
- `[=]`: 按值捕获所有外部变量
- `[&]`: 按引用捕获所有外部变量

Lambda 表达式常作为一些算法的参数, 如降序排序:

```
vector<int> nums={1, 4, 3, 5, 2, 6};  
sort(nums.begin(), nums.end(), [](int a, int b){return a >  
b;});
```

2.5 匿名函数

下面这段代码展示了按值捕获和按引用捕获的区别:

```
int x=1;
auto f=[x]() {cout << x << endl;};
f();
auto g=[&x]{x++;};
g();
cout << x << endl;
```

- 上方的写法有点函数式编程的味道, 把函数(对象)赋给了一个变量
- Lambda 表达式本质上是一个匿名的**函数对象**, 其类型由编译器生成, 每个 Lambda 都有独一无二的类型

3. 构造与析构

3.1 何为构造?

构造(Construction)不仅仅是“创建”一个对象，更是一个从无到有、从混乱到有序的过程。

回顾一下 C 语言动态分配内存:

```
int* p;  
p = malloc(5*sizeof(int));  
// do something  
free(p);
```

3.1 何为构造？

此处不详细讨论 `malloc` 的具体机制，这个在内存池(memory pool)实验中会涉及，而且内存管理实际很复杂。

忽略中间过程，假设我们已经通过 `malloc` 得到了一段内存，那么这段内存是什么样的呢？它是**未初始化的**，内容是随机的（`malloc` 才会将内存清零）。

构造的核心任务便是把这块原始内存按照你定义的逻辑进行初始化，使其成为一个合法的、可用的对象。

3.2 为什么需要构造？

如果没有构造过程，程序员必须在每次创建对象后手动给每个成员变量赋值。这会导致两个严重问题：

- 安全隐患：一旦忘记初始化，程序就会读到内存中的随机值，导致难以排除的 Bug
- 破坏封装：外部代码需要了解内部的实现细节才能正确设置初始状态

构造函数(Constructor)将初始化逻辑封装在类内部，确保对象“出身即合法”

3.3 构造函数

构造函数是一种特殊的成员函数，用于在创建对象时初始化对象的成员变量。它的名字必须与类名完全相同，且没有返回值（甚至不是 void）

如果不带参数，或者参数都有默认值，则称为默认构造函数：

```
class Player {  
public:  
    Player() {  
        score = 0;  
        cout << "Default Constructor" << endl;  
    }  
    int score;  
};
```

3.3 构造函数

- 构造时如果什么参数都不传，则会调用默认构造函数

```
Player p;  
cout << p.score << endl;
```

- 如果没有任何构造函数，编译器会自动生成一个默认构造函数，但对于内置类型（如 `int`、指针）的成员，其值是**未定义**的（随机值）
- 构造函数支持重载，如果定义了任意构造函数，编译器便不再自动生成默认构造函数

3.4 初始化列表

```
class Player{  
    Player()=default; // 提示编译器生成默认构造函数  
    Player(int score):score(score){}  
    int score;  
};
```

上方的代码中，并没有在构造函数函数体内赋值，而是使用了名为初始化列表(Initialization List)的语法。初始化列表直接构造成员变量，而函数体内赋值是先默认构造再赋值，因此初始化列表效率更高。对于 `const` 成员和引用成员，这是**唯一**的初始化方法（它们无法被赋值）。

3.5 析构函数

析构函数与构造函数对应，负责在对象生命周期结束时被自动调用，它的主要职责是释放对象在生存期内获取的资源（如堆内存、文件句柄、数据库连接等）

析构函数的名称与类名称完全相同，但在前面加上一个波浪号~。析构函数具备如下特性：

- 没有参数
- 没有返回值
- 不支持重载

3.5 析构函数

```
class Buffer {  
public:  
    Buffer(int size=0):size(size) {  
        data=new int[size];  
    }  
    ~Buffer() {  
        delete[] data; // 析构函数：释放资源  
        cout << "Destructor" << endl;  
    }  
    int* data;  
    int size;  
};
```

3.5 析构函数

析构函数在以下几种情况下会被自动触发:

1. 局部变量: 当程序离开对象作用域时 (例如函数结束)
2. 动态分配的对象: 当对指向的对象调用 `delete` 时
3. 静态/全局对象: 当整个程序结束时

```
// C++ 以花括号表示作用域
cout << "-----" << endl;
{
    Buffer buffer;
}
cout << "-----" << endl;
```

4. C++的内存管理

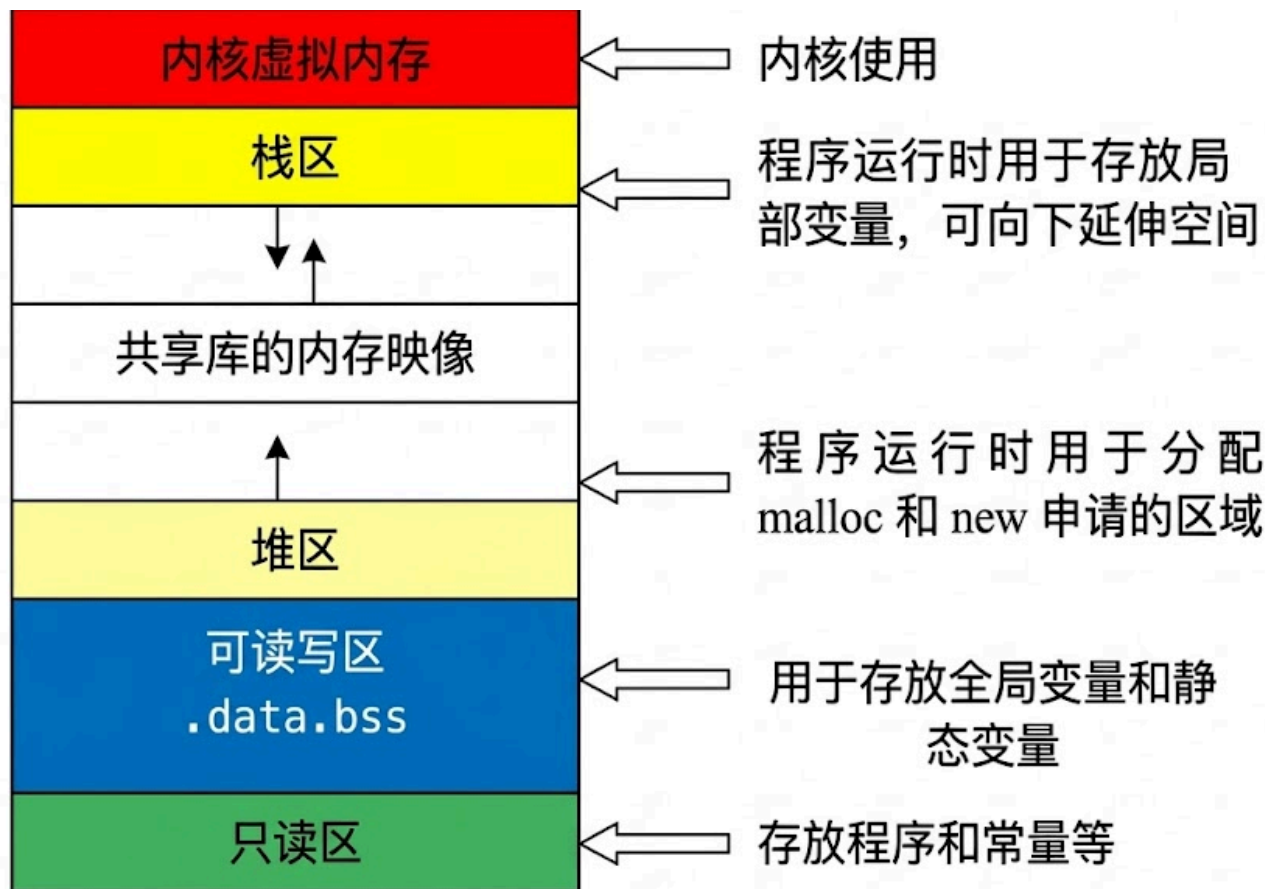
4.1 new 和 delete

遵循这种思想，C++提供了新的关键字 `new`，它在分配内存的同时会自动调用构造函数初始化对象。与之对应，`delete` 关键字用于销毁 `new` 出来的对象（会调用析构函数并释放内存）。注意 `malloc/free` 和 `new/delete` 不能交叉使用。

```
auto* p1=new Buffer();  
delete p1;
```

```
// 注意针对数组的用法  
int* p2=new int[4];  
delete[] p2;
```

4.2 内存分布



RAII(Resource Acquisition Is Initialization/资源获取即初始化)是 C++ 关于资源管理的编程范式，其核心思想是利用对象的生命周期管理资源：在构造函数中获取资源，在析构函数中释放资源，从而避免资源泄漏。其最典型的应用场景是动态内存分配：

- 构造对象：分配了内存
- 对象析构：释放了内存

这样能保证内存一定被释放，哪怕是中途执行出错（因为执行出错退出，也是离开作用域）

4.4 内存泄漏

上节课提到过 C++ 的内存泄漏是个老大难的问题，这里简单介绍下内存泄漏的检测方法：

- 静态检测（发现不匹配的 `new/delete`），如使用 `Cppcheck`
- 动态监测：
 - Valgrind: Linux 上的首选，其中的 Memcheck 模块可以检测所有内存相关错误，但是检测时程序性能会大幅下降
 - AddressSanitizer: 由 Google 开发的快速内存错误检测器，直接集成在了编译器中

5. 拷贝

5.1 浅拷贝 vs. 深拷贝

浅拷贝(Shallow Copy): 逐个成员进行二进制拷贝, 如果类成员中有指针, 浅拷贝只会复制指针地址, 导致两个对象指向同一块内存。

// 继续使用上面的Buffer作为例子

```
Buffer m1;
```

```
Buffer m2;
```

```
m2.data=m1.data;
```

```
for(int i=0;i<10;i++){m1.data[i]=i;}
```

```
for(int i=0;i<10;i++){cout << m2.data[i] << " " ;}
```

```
cout << endl;
```

// 程序退出时会触发 double free而崩溃

5.1 浅拷贝 vs. 深拷贝

深拷贝(Deep Copy): 不仅拷贝成员变量, 还会为指针成员重新分配一块内存, 并将原内存的内容复制过去

```
Buffer m1;  
Buffer m2;  
for(int i=0;i<10;i++){  
    m2.data[i]=m1.data[i];  
}  
for(int i=0;i<10;i++){m1.data[i]=i;}  
for(int i=0;i<10;i++){cout << m2.data[i] << " ";}  
cout << endl;
```

5.2 拷贝构造

拷贝构造函数用于使用一个现存对象初始化另一个对象（务必确保是深拷贝），其函数签名如下：

```
// ClassName 表示一个类名  
ClassName(const ClassName& other);
```

还是以上方的 Buffer 为例：

```
Buffer(const Buffer& other):size(other.size) {  
    data=new int[size];  
    for(int i=0;i<size;i++){  
        data[i]=other.data[i];  
    }  
}
```

5.2 拷贝构造

拷贝构造函数的触发场景如下:

1. `ClassA obj2 = obj1`, 注意此处不是调用拷贝赋值
2. `ClassA obj2(obj1);`
3. 以值传递方式向函数传对象

5.3 拷贝赋值运算符

拷贝赋值运算符用于将一个已存在对象的值赋给另一个已存在对象，其函数签名如下：

```
ClassName& operator=(const ClassName& other);
```

这实际上是运算符重载，运算符重载的内容后续课程会涉及，此处先直接记忆这个函数签名。

5.3 拷贝赋值运算符

与拷贝构造不同，被赋值对象已经持有资源，需要先释放旧资源再深拷贝：

```
Buffer& operator=(const Buffer& other) {  
    if (this == &other) return *this; // 防止自赋值  
    delete[] data;                     // 释放旧资源  
    size = other.size;  
    data = new int[size];  
    for (int i = 0; i < size; i++) data[i] = other.data[i];  
    return *this;  
}
```

5.4 this 指针

this 指针隐含于每一个非静态成员函数，它是一个极为特殊的指针，始终指向正在调用该函数的对象本身。以刚才的 Buffer 类拷贝构造函数为例，可以写成：

```
Buffer(const Buffer& other):size(other.size) {  
    this->data=new int[size];  
    for(int i=0;i<this->size;i++){  
        this->data[i]=other.data[i];  
    }  
}
```

简单来说，成员函数中所有对成员变量的访问均会被隐式添加 this 指针，不过你也可以显式写出以提高代码可读性。

5.4 this 指针

- `this` 指针实际是作为成员函数的第一个参数传入（如果你学过 Python 应该一下就能明白，因为在 Python 中要显式写出 `self`）
- `this` 的类型实际是 `ClassName* const`（指针本身不可修改）
- `this` 指针在成员函数开始执行前被创建，在函数执行后销毁